

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automatics

Tacu Igor — FAF-242

REPORT

Laboratory Work No. 1

Cryptography

The Caesar Cipher and the Caesar Cipher with a Permutation

Lecturer:
Maia Zaica
DISA, FCIM, UTM

Chişinău — 2026

Contents

1	Short Theory	2
1.1	The Caesar cipher	2
1.2	The Caesar cipher with a permutation	2
2	Implementation	3
2.1	Letter encoding	3
2.2	Input validation	3
2.3	Encryption and decryption (Task 1.1)	4
2.4	Permuted alphabet and two keys (Task 1.2)	4
2.5	Verification	5
3	Results	5
3.1	Task 1.1 – Caesar cipher	5
3.2	Task 1.2 – Caesar cipher with two keys	6
4	Conclusions	7
5	Source Code	7

1 Short Theory

1.1 The Caesar cipher

The Caesar cipher replaces every letter of the plaintext by the letter found k positions further in the alphabet, wrapping around at the end. With an alphabet of n letters whose positions are numbered from 0, the secret key is $k \in \{1, 2, \dots, n - 1\}$ and the same k is used for both operations:

$$\begin{aligned}c &= e_k(x) = (x + k) \bmod n, \\m &= d_k(y) = (y - k) \bmod n.\end{aligned}$$

When $y - k$ is negative, n must be added to bring the result back into the range $0 \dots n - 1$, e.g. $(1 - 3) \bmod 26 = 24$. For the Romanian alphabet of this work $n = 31$, so $k \in \{1, \dots, 30\}$.

Key space: only $n - 1 = 30$ keys. The cipher is broken by an *exhaustive search*: the attacker decrypts with every key and keeps the one that yields readable text.

1.2 The Caesar cipher with a permutation

To enlarge the key space, the alphabet is first permuted using a *keyword* k_2 : the distinct letters of k_2 are written first (only the first occurrence of each is kept), followed by the remaining letters of the alphabet in their natural order. The Caesar shift k_1 is then applied to the *positions in the permuted alphabet*. For $k_2 = \text{cryptography}$ and the English alphabet the order becomes CRYPTOGAHBDEFIJKLMNOPQSUUVWXZ, so with $k_1 = 3$ the text CIFRULCEZAR is encrypted as PLKTXQPJYDT (instead of FLIUXOFHCDU with the plain cipher). Encryption and decryption keep the same formulas, but x and y are now positions in the permuted alphabet.

Key space: there are $n!$ orderings and $n - 1$ shifts. For $n = 31$:

$$31! \times 30 \approx 8.22 \cdot 10^{33} \times 30 \approx 2.47 \cdot 10^{35} \quad (\approx 2^{117.6}),$$

which makes exhaustive search infeasible. However, a keyword always yields a *fixed* substitution, so letter frequencies of the language remain visible in the ciphertext and the cipher is still vulnerable to *frequency analysis*. The number of distinct encryptions cannot exceed $n!$, because different key pairs can produce the same substitution.

2 Implementation

The program is written in Python 3 (no external libraries) and consists of `caesar.py` (cipher + console interface) and `test_caesar.py` (unit tests). The alphabet is a parameter of every function, which allowed me to verify the code against the English examples from the assignment.

2.1 Letter encoding

```
1 ROMANIAN = ["A", "Ă", "Â", "B", "C", "D", "E", "F", "G", "H", "I",  
    , "Î", "J", "K", "L",  
2           "M", "N", "O", "P", "Q", "R", "S", "Ș", "T", "Ț", "U",  
    , "V", "W", "X", "Y", "Z"]  
3 CEDILLA = {"Ș": "Ș", "ș": "ș", "Ț": "Ț", "ț": "ț"}
```

The code of a letter is simply its index in this list, which reproduces Table 2 of the assignment (A = 0, Ă = 1, ..., Z = 30). Built-in character codes (`ord`, `chr`, ASCII/Unicode) are *never* used to compute the shift. `CEDILLA` maps the cedilla letters Ș, Ț to the comma-below letters so that text typed on different keyboards is accepted.

2.2 Input validation

```
1 def normalize(text, alphabet=ROMANIAN):  
2     out = ""  
3     for ch in text.replace(" ", ""):  
4         ch = CEDILLA.get(ch, ch).upper()  
5         if ch not in alphabet:  
6             raise InvalidInput(f"Invalid character '{ch}'.  
7                 Allowed: ...")  
8         out += ch  
9     return out  
10  
11 def validate_shift(raw, n=len(ROMANIAN)):  
12     try:  
13         k = int(raw)  
14     except ValueError:  
15         k = None  
16     if k is None or not 1 <= k < n:  
17         raise InvalidInput(f"Invalid key '{raw.strip()}'. The key  
    must be an "  
    f"integer between 1 and {n - 1}  
    inclusive.")
```

```
18     return k
```

`normalize` removes spaces, unifies cedilla variants, converts to uppercase (requirement 6) and rejects the first character that is not in the alphabet, naming it in the message. `validate_shift` accepts only integers in $1 \dots 30$. `validate_keyword` (not shown) applies `normalize` to the keyword and additionally rejects keywords with spaces or shorter than 7 letters. All validators raise `InvalidInput`; the function `ask` catches it, prints the message and asks for the value again:

```
1 def ask(prompt, validate):
2     while True:
3         try:
4             return validate(input(prompt))
5         except InvalidInput as err:
6             print(f"    [!] {err}")
```

2.3 Encryption and decryption (Task 1.1)

```
1 def shift(text, k, alphabet):
2     n = len(alphabet)
3     pos = {ch: i for i, ch in enumerate(alphabet)}
4     return "".join(alphabet[(pos[ch] + k) % n] for ch in text)
5
6 def encrypt(text, k, alphabet=ROMANIAN):    #  $c = (x + k) \bmod n$ 
7     return shift(text, k, alphabet)
8
9 def decrypt(text, k, alphabet=ROMANIAN):    #  $m = (y - k) \bmod n$ 
10    return shift(text, -k, alphabet)
```

`pos` maps each letter to its position and `shift` applies the formulas of Section 1; decryption is the same operation with $-k$. Python's `%` always returns a value in $0 \dots n - 1$, so a negative $y - k$ is wrapped around correctly (e.g. $(1 - 3) \bmod 31 = 29$).

2.4 Permuted alphabet and two keys (Task 1.2)

```
1 def permuted_alphabet(keyword, alphabet=ROMANIAN):
2     # dict keeps insertion order: repeats dropped, first
3     # occurrence kept
4     return list(dict.fromkeys(keyword + "".join(alphabet)))
5
6 def encrypt2(text, k, keyword, alphabet=ROMANIAN):
7     return encrypt(text, k, permuted_alphabet(keyword, alphabet))
```

Putting the keyword in front of the whole alphabet and removing repeated letters, produces exactly the construction from the assignment (first occurrence of each keyword letter, then the remaining letters in natural order). The resulting order simply replaces the alphabet in `encrypt/decrypt`, so the second task reuses the code of the first. The program prints the permuted alphabet before every operation so that the result can be checked by hand.

2.5 Verification

`test_caesar.py` checks the code against all worked examples of the assignment (using the English alphabet): `CIFRULCEZAR` $\rightarrow$ `FLIUXOFHCDU` ($k = 3$), `BRUTEFORCEATTACK` $\rightarrow$ `SILKVWFITVRKKRTB` ($k = 17$) together with the brute-force candidate for $k = 1$, the permuted alphabet for *cryptology*, and `CIFRULCEZAR` $\rightarrow$ `PLKTXQPJYDT` with $k_1 = 3$, $k_2 = \text{cryptology}$. For the Romanian alphabet it checks the size (31) and codes, the wrap-around cases, round trips for all 30 keys, and the rejected inputs. All 9 tests pass (`python3 -m unittest`).

3 Results

The listings below are the real output of the program (the text typed by the user is shown after each prompt).

3.1 Task 1.1 – Caesar cipher

```
Choice: 1
Operation - (e)ncrypt or (d)ecrypt: e
Key 1 (shift, 1-30): 3
Enter the plaintext: cifrul cezar
Normalized input : CIFRULCEZAR
Ciphertext       : FKITXOFHÂBT

Choice: 1
Operation - (e)ncrypt or (d)ecrypt: d
Key 1 (shift, 1-30): 3
Enter the ciphertext: FKITXOFHÂBT
Normalized input : FKITXOFHÂBT
Plaintext       : CIFRULCEZAR
```

Listing 1: Encryption and decryption with $k = 3$.

The result differs from `FLIUXOFHCDU` of the English example because the Romanian alphabet has 31 letters including Ă, Â, Î, Ș, Ț: for example $C = 4 \rightarrow 7 = F$ and $I = 10 \rightarrow 13 = K$.

```

Key 1 (shift, 1-30): 0
[!] Invalid key '0'. The key must be an integer between 1 and 30
    inclusive.
Key 1 (shift, 1-30): 31
[!] Invalid key '31'. The key must be an integer between 1 and 30
    inclusive.
Key 1 (shift, 1-30): abc
[!] Invalid key 'abc'. The key must be an integer between 1 and 30
    inclusive.
Key 1 (shift, 1-30): 5
Enter the plaintext: hello, world
[!] Invalid character ', '. Allowed: letters of the Romanian alphabet (
    A-Z, Ă, Â, Î, Ș, Ț) and spaces.
Enter the plaintext: Știință și artă
Normalized input : ȘTIINȚĂȘIARTĂ
Ciphertext       : WXMSYEWMDUXE

```

Listing 2: Task 1.1 – invalid key and invalid character.

The last line shows the cedilla letters being accepted as Ș and Ț.

3.2 Task 1.2 – Caesar cipher with two keys

```

Choice: 2
Operation - (e)ncrypt or (d)ecrypt: e
Key 1 (shift, 1-30): 3
Key 2 (keyword, min 7 letters): criptografie
Enter the plaintext: Cifrul Cezar
Permuted alphabet: C R I P T O G A F E Ă Â B D H Î J K L M N Q S Ș Ț U V
    W X Y Z
Normalized input : CIFRULCEZAR
Ciphertext       : POÂTXQPBIĂT

Choice: 2
Operation - (e)ncrypt or (d)ecrypt: d
Key 1 (shift, 1-30): 3
Key 2 (keyword, min 7 letters): criptografie
Enter the ciphertext: POÂTXQPBIĂT
Permuted alphabet: C R I P T O G A F E Ă Â B D H Î J K L M N Q S Ș Ț U V
    W X Y Z
Normalized input : POÂTXQPBIĂT
Plaintext        : CIFRULCEZAR

```

Listing 3: Encryption and decryption with $k_1 = 3$, $k_2 = \text{criptografie}$.

Check by hand: C has position 0 in the permuted alphabet, so it becomes the letter at position 3, which is P; I (position 2) becomes the letter at position 5, O.

```
Key 2 (keyword, min 7 letters): cripto
[!] Keyword too short (6 letters). It needs at least 7.
Key 2 (keyword, min 7 letters): cripto grafie
[!] Invalid keyword. It must be a single word without spaces.
Key 2 (keyword, min 7 letters): cripto9rafie
[!] Invalid character '9'. Allowed: letters of the Romanian alphabet (
    A-Z, Ă, Â, Î, Ș, Ț) and spaces.
Key 2 (keyword, min 7 letters): criptografie
```

Listing 4: Task 1.2 – invalid keywords.

4 Conclusions

The plain Caesar cipher has only $n - 1 = 30$ keys for the Romanian alphabet, so an exhaustive search finds the plaintext immediately; it offers no real protection. Using a keyword to permute the alphabet raises the number of keys to about $31! \cdot 30 \approx 2.5 \cdot 10^{35}$ and makes brute force infeasible, but the cipher remains a monoalphabetic substitution: every plaintext letter is always replaced by the same ciphertext letter, so a frequency analysis of a sufficiently long ciphertext still reveals the substitution. A large key space is therefore necessary but not sufficient for security.

During the work I learned to implement modular arithmetic with explicit handling of negative values, to build the permuted alphabet from a keyword, and to validate user input with clear error messages. Two practical details mattered: the Romanian alphabet must be encoded from the given table (not from Unicode code points, which are not contiguous for Ă, Â, Î, Ș, Ț), and the cedilla/comma-below variants of Ș and Ț must be unified. Testing the alphabet-independent functions against the English examples of the assignment gave confidence that the Romanian version is correct.

5 Source Code

The full source code is available in the GitHub repository:

<https://github.com/igortacu/cryptography>